

Bootcamp: Full Stack Junior

Modulo 4: Profesionalización Fronted con TypeScript y Next.js

Semana 12

Contenido

1

App Router y sistema de navegación.

2

Estrategias de renderizado: SSR, CSR y Static Generation.

3

React Server Components (RSC) y optimización de Assets.

Contenido:

1. Introducción a Next.js (The React Framework for the Web)

React es una **librería**: te da herramientas para construir interfaces (componentes, hooks, estado), pero **no decide por ti** cómo estructurar la app, cómo hacer routing, SEO o renderizado en servidor. Eso te da libertad... pero también complejidad.

Next.js es un **framework** construido **encima de React**. Un framework **impone convenciones**, trae decisiones tomadas y una arquitectura lista para producción. El resultado: menos configuración manual y más foco en construir producto.

En proyectos reales (startups, SaaS, e-commerce), usar React “puro” suele implicar agregar muchas librerías externas. Next.js integra todo eso de forma nativa y optimizada.

Ventajas clave de Next.js

- SEO real: páginas renderizadas en servidor (SSR) o pre-generadas (SSG).
- Mejor rendimiento: contenido visible desde el primer byte.
- Routing automático: sin React Router.
- Optimización de imágenes y fuentes integrada.
- Escalable: pensado para producción desde el día uno.



Ilustración 1 - Nextjs - Fuente: Nextjs

Tabla comparativa: React (Vite) vs Next.js

Característica	React + Vite	Next.js
Tipo	Librería	Framework
SEO	Limitado (CSR)	Excelente (SSR / SSG)
Routing	React Router	File-based automático
Renderizado	Solo cliente	Cliente + Servidor

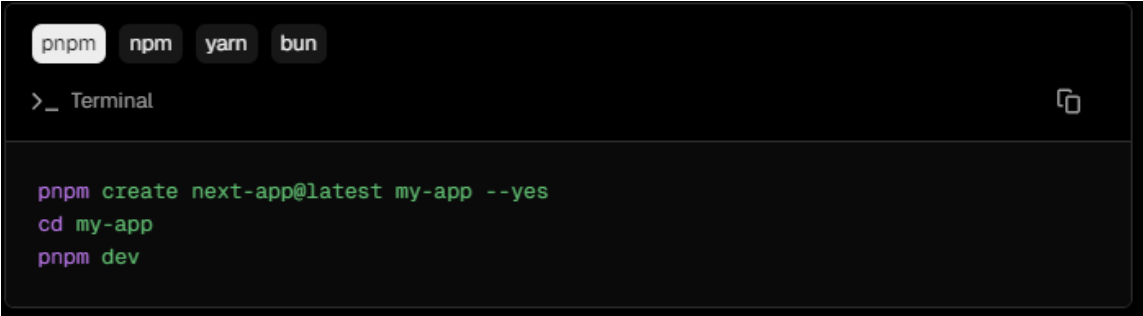
Optimización imágenes	Manual	Automática
Configuración inicial	Media	Media
Producción real	Requiere extras	Listo desde inicio

Tabla 1 - React vs Nextjs - Fuente: Autor de Contenidos

1.1. Instalación y Estructura del Proyecto

Instalación oficial

Next.js se instala con un **asistente interactivo** que define la base del proyecto profesional.



```
pnpm create next-app@latest my-app --yes
cd my-app
pnpm dev
```

Ilustración 2 - Inicialización de proyecto con Nextjs - Fuente: Nextjs

Durante la instalación, Next.js te preguntará:

- **TypeScript** - recomendado (industria real)
- **ESLint** - calidad de código
- **Tailwind CSS** - opcional pero muy usado
- **App Router** - estándar actual
- **Alias @/** - imports limpios

Esto evita decisiones incorrectas comunes en proyectos junior.

Estructura de proyecto

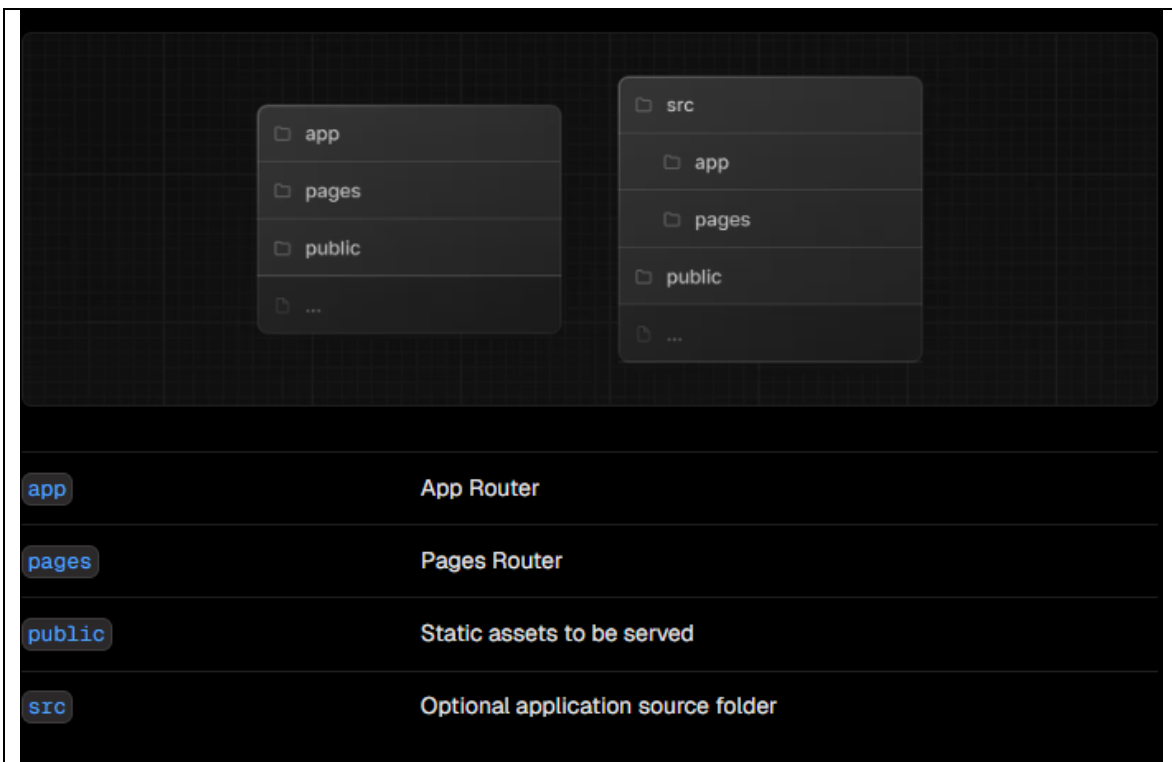


Ilustración 3 -Estructura de proyecto con Nextjs - Fuente: Nextjs

1.2. El gran cambio - Pages Router vs App Router

Antes de Next 13, todo vivía en la carpeta `pages/`. Funcionaba bien, pero tenía límites para layouts complejos y rendimiento.

El **App Router** nace para alinearse con **React Server Components**, permitiendo:

- Menos JavaScript en el cliente
- Mejor performance
- Layouts anidados reales

Hoy, **App Router** es el estándar oficial.

Aspecto	Pages Router	App Router
Carpeta	pages/	app/
Layouts	Limitados	Nativos y anidados
Server Components	No	Si
Escalabilidad	Media	Alta
Futuro	Legacy	Estándar

Tabla 2 - Pages Router vs App Router - Fuente: Autor de Contenidos

2. Anatomía del App Router (File-based Routing)

2.1. ¿Cómo funcionan las rutas en Next.js 16?

En el **App Router**, las rutas **no se configuran**, se **descubren**. Cada carpeta dentro de `/app` representa una parte de la URL, y ciertos archivos especiales definen **qué se renderiza** y **cómo se comporta** la ruta.

Esto significa que la estructura del proyecto **es el mapa mental de tu aplicación**. Cuando un desarrollador ve el árbol de carpetas, entiende inmediatamente las URLs disponibles. Esta convención reduce errores, elimina boilerplate y mejora la mantenibilidad del proyecto.

Además, en Next.js 16:

- Los **Server Components** son el estándar por defecto.
- El **Streaming** permite enviar HTML al navegador por partes, mejorando el TTFB (Time To First Byte).

Archivos especiales del App Router

Archivo	Función
page.tsx	Define el contenido principal de una ruta
layout.tsx	Layout persistente que envuelve rutas hijas
not-found.tsx	Página personalizada para errores 404
loading.tsx	UI de carga durante streaming
error.tsx	Manejo de errores en runtime
route.ts	Endpoints tipo API (Route Handlers)

Tabla 2 - App Router - Fuente: Nextjs

Ejemplo de estructura con rutas dinámicas

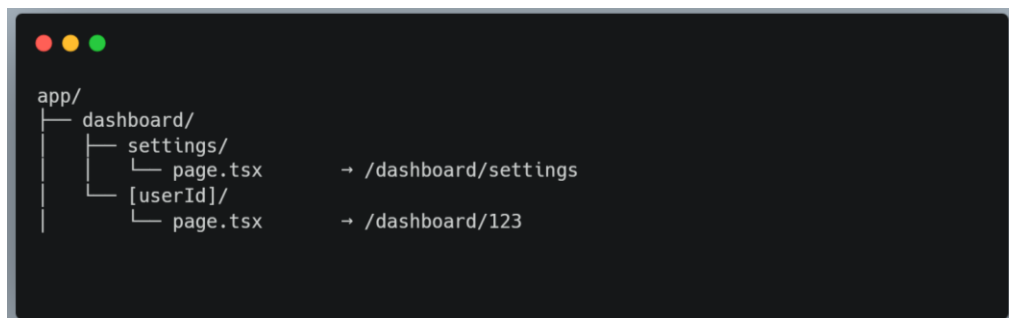


Ilustración 4 - Manejo de rutas en Nextjs - Fuente: Nextjs

Si dudas sobre una URL, **mírate la carpeta**. Next.js siempre sigue la convención, nunca la rompe.

2.2. Navegación entre Rutas

Navegación declarativa con <Link>

El componente <Link> de Next.js **no es un <a> normal**. Está optimizado para:

- **Prefetch automático** de rutas
- Navegación instantánea
- Mejor UX sin recargas

Siempre que renderices enlaces visibles al usuario, **usa <Link>**.



```
import Link from 'next/link'

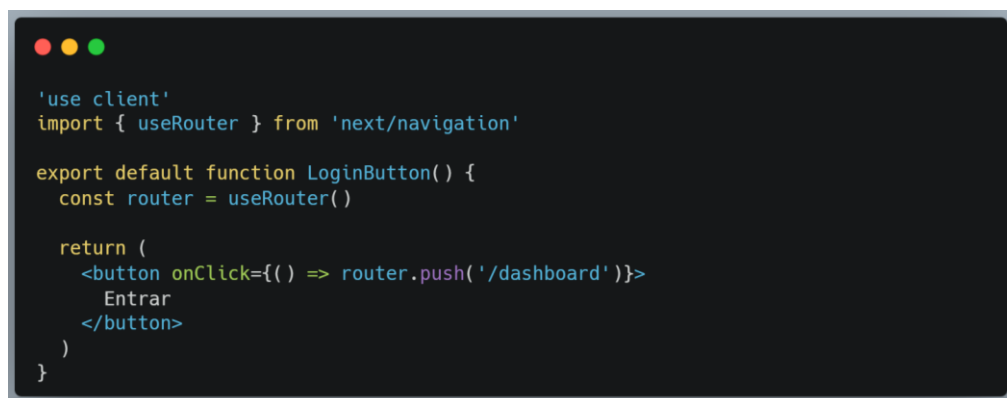
export default function Nav() {
  return (
    <nav className="flex gap-4">
      <Link href="/dashboard">Dashboard</Link>
      <Link href="/settings">Settings</Link>
    </nav>
  )
}
```

Ilustración 5 -Ejemplo de navegación entre páginas en Nextjs - Fuente: Autor de contenidos

Navegación programática

Cuando necesitas navegar por lógica (submit, auth, eventos), tienes dos opciones:

Client Side: useRouter



```
'use client'
import { useRouter } from 'next/navigation'

export default function LoginButton() {
  const router = useRouter()

  return (
    <button onClick={() => router.push('/dashboard')}>
      Entrar
    </button>
  )
}
```

Ilustración 6 -Ejemplo de navegación CSR Nextjs - Fuente: Autor de contenidos

Server Side: redirect

```

import { redirect } from 'next/navigation'

export default function Page() {
  const isAuthenticated = false

  if (!isAuthenticated) {
    redirect('/login')
  }

  return <h1>Dashboard</h1>
}

```

Ilustración 7 -Ejemplo de navegación SSR Nextjs - Fuente: Autor de contenidos

¿Cuándo usar cada uno?

Escenario	Herramienta
Enlaces visibles	<Link>
Eventos de usuario	useRouter()
Auth / guards	redirect()
Server Components	redirect()
Client Components	useRouter()

Tabla 3 – Métodos de uso de navegación en Nextjs - Fuente: Nextjs

3. Layouts vs Templates vs Route Groups

3.1. Layouts: persistencia y rendimiento

Un `layout.tsx` **no se desmonta** al cambiar de página dentro de su jerarquía.

Esto permite:

- Mantener estado
- Evitar re-renders innecesarios
- Mejor rendimiento

Además, es el lugar ideal para **SEO y metadatos**.

```
// app/dashboard/layout.tsx
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: 'Dashboard',
  description: 'Panel principal del usuario',
}

interface Props {
  children: React.ReactNode
}

export default function DashboardLayout({ children }: Props) {
  return (
    <section>
      <aside>Sidebar</aside>
      <main>{children}</main>
    </section>
  )
}
```

Ilustración 8 -Layout Nextjs - Fuente: Autor de contenidos

Templates: forzar re-montado

Un `template.tsx` se vuelve a montar en cada navegación.

Útil para:

- Animaciones de entrada
- Resetear estado
- Wizards o flows temporales

Característica	Layout	Template
Mantiene estado	Si	No
Re-monta	No	Si
Rendimiento	Alto	Medio
Uso común	UI persistente	Animaciones

Tabla 4 – Layout vs templates - Fuente: Autor de contenidos

4. Estrategias de Renderizado en Next.js 16+

4.1. Del HTML estático al tiempo real

En la web moderna no existe una sola forma “correcta” de renderizar. Next.js 16 te da un **espectro** que va desde lo **ultra rápido y estático** hasta lo **dinámico en tiempo real**. La clave está en **elegir la estrategia correcta según el tipo de dato y la experiencia de usuario**.

Un concepto fundamental aquí es la **hidratación (Hydration)**. Significa que el HTML

generado en el servidor llega al navegador y luego React “lo despierta” para hacerlo interactivo. En Next.js 16, esta hidratación ocurre **solo donde hace falta**, gracias a Server Components.

Lo poderoso es que **puedes mezclar estrategias**:

- Un layout estático
 - Una lista renderizada en servidor
 - Un botón interactivo en cliente
- Todo en la misma página.

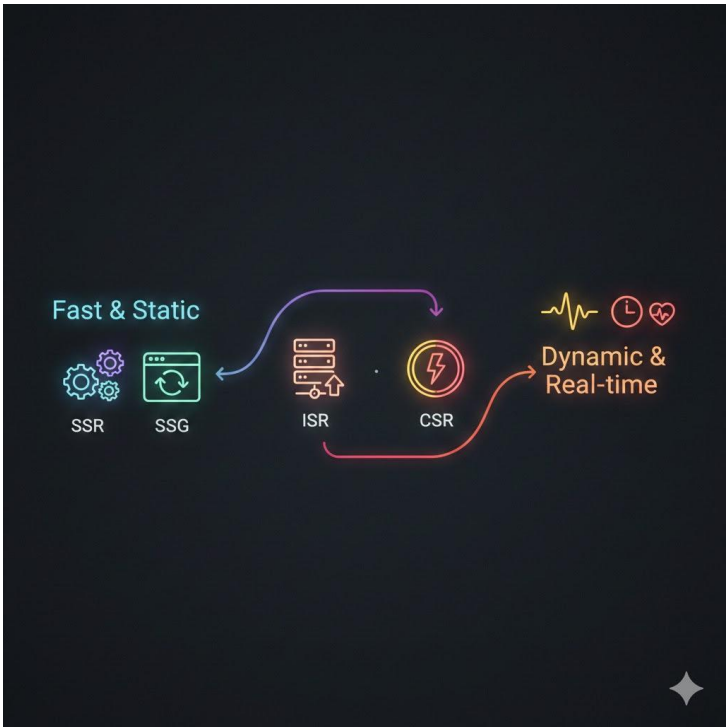


Ilustración 9 -Estrategias de renderizado - Fuente: Gemini nano banana

Estrategia	Velocidad	SEO	Frescura de datos	Uso típico
CSR	Media	Baja	Alta	Dashboards
SSR	Media	Alta	Muy alta	Datos en tiempo real
SSG	Muy alta	Excelente	Baja	Blogs
ISR	Muy alta	Excelente	Media	E-commerce

Tabla 5 – Comparativa de métodos de renderizado - Fuente: Autor de contenidos

4.2. Client-Side Rendering (CSR) — Interactividad pura

'use client'

En Next.js 16, **todo es Server Component por defecto**. Solo usas 'use client' cuando

necesitas:

- Hooks (useState, useEffect)
- Event listeners (onClick)
- APIs del navegador (localStorage)

CSR **no es malo**, simplemente es **interactividad**. El error común es usarlo en exceso.

A screenshot of a code editor with a dark background. It shows a React component named Counter. The code uses 'use client' at the top, imports 'useState' from 'react', and defines a function Counter() that uses useState(0) to manage a count. It returns a button with Tailwind CSS classes and an onClick handler that increments the count. The button also displays the current count.

```
'use client'
import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)

  return (
    <button
      className="px-4 py-2 bg-blue-600 text-white rounded"
      onClick={() => setCount(count + 1)}
    >
      Clicks: {count}
    </button>
  )
}
```

Ilustración 10 -Client Side Rendering - Fuente: Autor de contenidos

Este componente:

- Se renderiza en el cliente
- Descarga JavaScript
- Se hidrata para ser interactivo

4.3. Server-Side Rendering (SSR) — Datos en tiempo real

SSR en Next.js 16

En Next.js 16 puedes hacer **fetch directamente en el componente**, sin hooks, sin `getServerSideProps`.

Si el dato **debe ser siempre fresco**, usas SSR.

Esto es como **comida hecha al momento**: tarda un poco más, pero llega caliente.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a light-colored monospace font. It defines a constant 'dynamic' with the value 'force-dynamic', an async function 'getData' that fetches data from 'https://api.example.com/data' with 'no-store' cache, and a default async function 'Page' that calls 'getData' and returns the data as a JSON stringified object.

```
export const dynamic = 'force-dynamic'

async function getData() {
  const res = await fetch('https://api.example.com/data', {
    cache: 'no-store',
  })
  return res.json()
}

export default async function Page() {
  const data = await getData()
  return <pre>{JSON.stringify(data, null, 2)}</pre>
}
```

Ilustración 11 -Server Side Rendering - Fuente: Autor de contenidos

Aquí:

- El servidor hace el fetch
- Genera HTML en cada request
- El usuario recibe la página lista

4.4. SSG e ISR — Rendimiento a escala

SSG

Static Site Generation genera el HTML **en build time**.

Resultado: páginas **ultra rápidas** y con **SEO perfecto**.

Ideal para contenido que **casi no cambia**.

ISR: Stale-While-Revalidate

ISR combina lo mejor de ambos mundos:

- Sirve una página estática
- La actualiza en segundo plano

5. Introducción a React Server Components (RSC)

5.1. ¿Qué son los React Server Components?

Los **React Server Components (RSC)** son componentes que **se ejecutan exclusivamente en el servidor**. Su resultado (HTML + payload mínimo) se envía al navegador, **sin incluir JavaScript innecesario** para el cliente.

Esto rompe el paradigma clásico de “todo es JavaScript en el navegador”. En su lugar:

- El **servidor hace el trabajo pesado**
- El **cliente solo recibe lo que necesita**
- La app es **más rápida, más ligera y más escalable**

En Next.js 16 (App Router), **todos los componentes son Server Components por defecto**. Solo pasas a Client Components cuando realmente necesitas interactividad.

5.2. ¿Qué son los React Server Components?

Los beneficios clave son:

- **Menor bundle size:** menos JS descargado → mejor performance.
- **Lógica en el servidor:** puedes usar `async/await` directamente.
- **Acceso directo a datos:** bases de datos, servicios internos, secretos.
- **Más seguridad:** credenciales nunca llegan al cliente.

Característica	Server Components	Client Components
Renderizado	Servidor (Build o Request)	Navegador (Hydration)
Hooks (useState, useEffect)	No permitidos	Obligatorios
Acceso a datos	Directo (async/await)	Vía API / Fetch
Seguridad	Alta (server-only)	Menor
Directiva	Por defecto en App Router	'use client'
Uso típico	Data, layouts, páginas	Botones, formularios

Tabla 6 – Server Components vs Client Components - Fuente: Autor de contenidos

Recursos de Apoyo



Material de Lectura

[Comparación de estrategias de renderizado \(SSR, SSG, ISR, CSR\).](#)



Video

[Tutorial de cómo usar Nextjs SSR](#)



Curso

[Curso de app router](#)